Big Data and Automated Content Analysis (6EC)
Week 1: »Programming for Computational (Communication|Social) Scientists«
Thursday

Anne Kroon
a.c.kroon@uva.nl

October 29, 2026

UvA RM Communication Science

Today

Datatypes

Functions and methods

Modifying lists and dictionaries

for, if/elif/else, try/except

Optional: Python extras

Next steps

Datatypes

Python lingo

Basic datatypes (variables)

int `37`

float `1.75`

bool `True`, `False`

string `"Alice"`

(variable name `firstname`)

"firstname" and firstname is not the same.

"5" and 5 is not the same.

But you can transform it: `int("5")` will return 5.

You cannot calculate `3 * "5"` (In fact, you can. It's `"555"`).

But you can calculate `3 * int("5")`

Python lingo

Basic datatypes (variables)

int `37`

float `1.75`

bool `True`, `False`

string `"Alice"`

(**variable name** `firstname`)

"firstname" and firstname is not the same.

"5" and 5 is not the same.

But you can transform it: `int("5")` will return 5.

You cannot calculate `3 * "5"` (In fact, you can. It's `"555"`).

But you can calculate `3 * int("5")`

Python lingo

Basic datatypes (variables)

int `37`

float `1.75`

bool `True`, `False`

string `"Alice"`

(**variable name** `firstname`)

”firstname” and firstname is not the same.

”5” and 5 is not the same.

But you can transform it: `int("5")` will return 5.

You cannot calculate `3 * "5"` (In fact, you can. It’s `"555"`.

But you can calculate `3 * int("5")`

Python lingo

More advanced datatypes

```
list firstnames = ['Alice','Bob','Cecile']
     lastnames = ['Garcia','Lee','Miller']
list ages = [18,22,45]
dict agedict = {'Alice': 18, 'Bob': 22,
     'Cecile': 45}
```

Note that the elements of a list, the keys of a dict, and the values of a dict can have any* datatype! (You can even mix them, but it's better to be consistent!)

*Well, keys cannot be mutable → see book

Python lingo

More advanced datatypes

```
list firstnames = ['Alice','Bob','Cecile']
     lastnames = ['Garcia','Lee','Miller']

list ages = [18,22,45]

dict agedict = {'Alice': 18, 'Bob': 22,
     'Cecile': 45}
```

Note that the elements of a list, the keys of a dict, and the values of a dict can have any* datatype! (You can even mix them, but it's better to be consistent!)

*Well, keys cannot be mutable → see book

Python lingo

More advanced datatypes

```
list firstnames = ['Alice','Bob','Cecile']
     lastnames = ['Garcia','Lee','Miller']
list ages = [18,22,45]
dict agedict = {'Alice': 18, 'Bob': 22,
     'Cecile': 45}
```

Note that the elements of a list, the keys of a dict, and the values of a dict can have any* datatype! (You can even mix them, but it's better to be consistent!)

*Well, keys cannot be mutable → see book

Python lingo

More advanced datatypes

```
list firstnames = ['Alice','Bob','Cecile']
     lastnames = ['Garcia','Lee','Miller']
list ages = [18,22,45]
dict agedict = {'Alice': 18, 'Bob': 22,
     'Cecile': 45}
```

Note that the elements of a list, the keys of a dict, and the values of a dict can have any* datatype! (You can even mix them, but it's better to be consistent!)

*Well, keys cannot be mutable → see book

Python lingo

More advanced datatypes

```
list firstnames = ['Alice','Bob','Cecile']
     lastnames = ['Garcia','Lee','Miller']
list ages = [18,22,45]
dict agedict = {'Alice': 18, 'Bob': 22,
     'Cecile': 45}
```

Note that the elements of a list, the keys of a dict, and the values of a dict can have any* datatype! (You can even mix them, but it's better to be consistent!)

*Well, keys cannot be mutable → see book

Python lingo

Retrieving specific items

list `firstnames[0]` gives you the first entry

`firstnames[-2]` gives you the one-but-last entry

`firstnames[:2]` gives you entries 0 and 1

`firstnames[1:3]` gives you entries 1 and 2

`firstnames[1:]` gives you entries 1 until the end

dict `agedict["Alice"]` gives you 18

Python lingo

Retrieving specific items

list `firstnames[0]` gives you the first entry

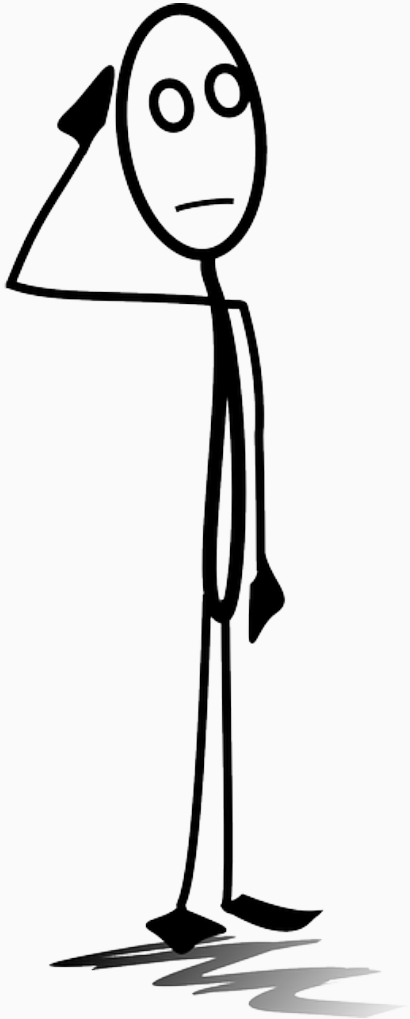
`firstnames[-2]` gives you the one-but-last entry

`firstnames[:2]` gives you entries 0 and 1

`firstnames[1:3]` gives you entries 1 and 2

`firstnames[1:]` gives you entries 1 until the end

dict `agedict["Alice"]` gives you 18



Think of at least two different ways of storing data about some fictious persons (first name, last name, age, phone number, ...) using lists and/or dictionaries. What are the pros and cons?

Python lingo

Less frequent, but still useful datatypes

set	A collection in which each item is unique: `{1,2,3}`
tuple	Like a list, but *immutable*: `(1,2,2,2,3)`
defaultdict	A dict that does not raise an error but returns the "empty" value of its datatype (0 for int, "" for str) if you try access a non-existing key (great for storing results and counting things!)
np.array	A list-like datatype provided by the `numpy` package optimized for efficient mathematical operations.
...	...

You will come across more later

Functions and methods

Python lingo

Functions

functions Take an input and return something else `int(32.43)` returns the integer 32. `len("Hello")` returns the integer 5.

methods are similar to functions, but directly associated with an object. `"SCREAM".lower()` returns the string "scream"

Both functions and methods end with `()`. Between the `()`, *arguments* can (sometimes have to) be supplied.

Python lingo

Functions

functions Take an input and return something else `int(32.43)` returns the integer 32. `len("Hello")` returns the integer 5.

methods are similar to functions, but directly associated with an object. `"SCREAM".lower()` returns the string "scream"

Both functions and methods end with `()`. Between the `()`, *arguments* can (sometimes have to) be supplied.

Python lingo

Functions

functions Take an input and return something else `int(32.43)` returns the integer 32. `len("Hello")` returns the integer 5.

methods are similar to functions, but directly associated with an object. `"SCREAM".lower()` returns the string "scream"

Both functions and methods end with `()`. Between the `()`, *arguments* can (sometimes have to) be supplied.

Python lingo

Functions

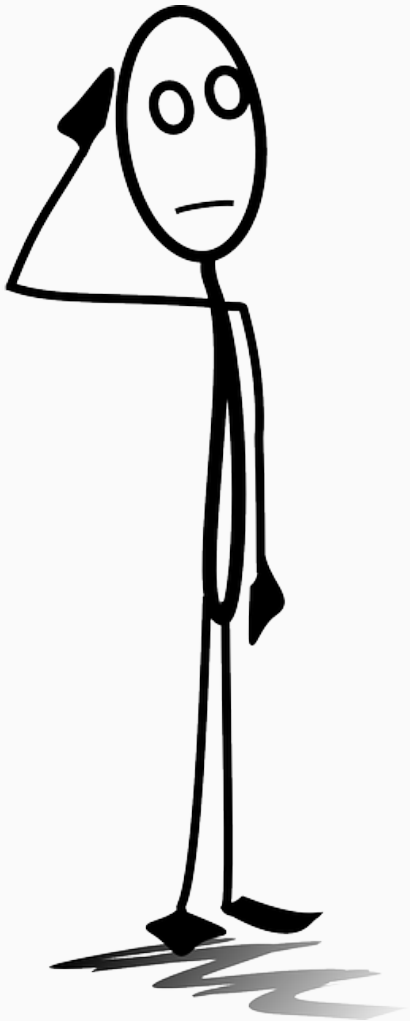
functions Take an input and return something else `int(32.43)` returns the integer 32. `len("Hello")` returns the integer 5.

methods are similar to functions, but directly associated with an object. `"SCREAM".lower()` returns the string "scream"

Both functions and methods end with `()`. Between the `()`, *arguments* can (sometimes have to) be supplied.

Some functions

```
1  len(x)          # returns the length of x
2  y = len(x)      # assign the value returned by len(x) to y
3  print(len(x)) # print the value returned by len(x)
4  print(y)        # print y
5  int(x)          # convert x to an integer
6  str(x)          # convert x to a string
7  sum(x)          # get the sum of x
```



How could you print the mean (average) of a list of integers using the functions on the previous slide?

Some methods

Some string methods

```
mystring = "Hi! How are you?"
mystring.lower()   # return lowercased string (doesn't change original!)
mylowercasedstring =  mystring.lower()  # save to a new variable
mystring =  mystring.lower()  # or override the old one
mystring.upper()   # uppercase
mystring.split()   # Splits on spaces and returns a list ['Hi!', 'How',
↪  'are', 'you?']
```

We'll look into some list methods later.

⇒ **You can use TAB-completion in Jupyter to see all methods (and properties) of an object!**

Writing own functions

You can write an own function:

```
1  def addone(x):
2      y = x + 1
3      return y
```

Functions take some input ("argument") (in this example, we called it `x`) and *return* some result. Thus, running

```
1  addone(5)
```

returns `6`.

Writing own functions

Attention, R users! (maybe obvious for others?)

You *cannot** apply the function that we just created on a whole list – after all, it takes an int, not a list as input.

(wait a sec foruntil we cover for loops later today, but this is how you'd do it (by calling the function for each element in the list separately):):

```
1 mynumbers = [5, 3, 2, 4]
2 results = [addone(e) for e in mynumbers]
```

* Technically speaking, you could do this by wrapping the `map` function around your own function, but that's not considered "pythonic". Don't do it ;-)

Modifying lists & dicts

Modifying lists

Let's use one of our first methods! Each *list* has a method `.append()`:

Appending to a list

```
1  mijnlijst = ["element 1", "element 2"]
2  anotherone = "element 3"   # note that this is a string, not a list!
3  mijnlijst.append(anotherone)
4  print(mijnlijst)
```

gives you:

```
1  ["element 1", "element 2", "element 3"]
```

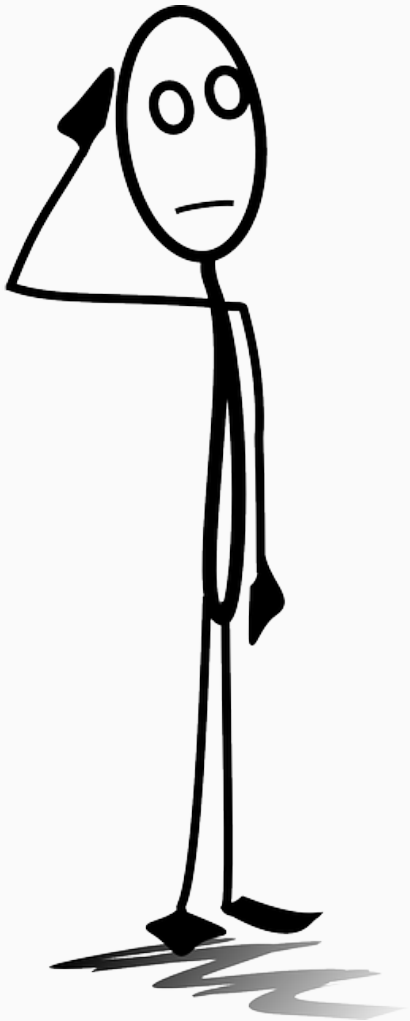
Modifying lists

Merging two lists (= extending)

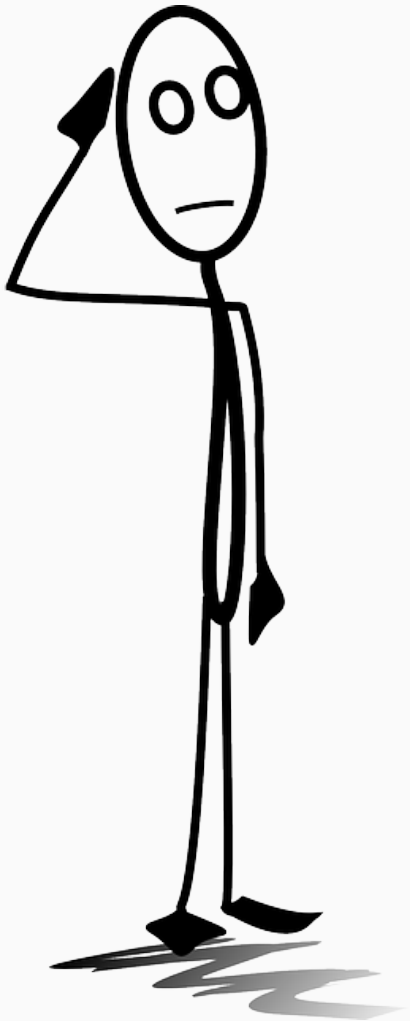
```
1  mijnlijst = ["element 1", "element 2"]
2  anotherone = ["element 3", "element 4"]
3  mijnlijst.extend(anotherone)
4  print(mijnlijst)
```

gives you:

```
1  ["element 1", "element 2", "element 3", "element 4"]
```



What would have happened if we had used `.append()` *instead of* `.extend()`*?*



Why do you think that the Python developers implemented `.append()` and `.extend()` as methods of a list and not as functions?

Modifying dicts

Adding a key to a dict (or changing the value of an existing key)

```
1  mydict = {"whatever": 42, "something": 11}
2  mydict["somethingelse"] = 76
3  print(mydict)
```

gives you:

```
1  {'whatever': 42, 'somethingelse': 76, 'something': 11}
```

If a key already exists, its value is simply replaced.

for, if/elif/else, try/except

How can we structure our program?

If we want to *repeat* a block of code, execute a block of code only *under specific conditions*, or more generally want to structure our code, we use *indention*.

Indention: The Python way of structuring your program

- Your program is structured by TABs or SPACEs.
- Jupyter (or your IDE) handles (guesses) this for you, but make sure to not interfere and not to mix TABs or SPACEs!
- Default: four spaces per level of indention.

Indention

Structure

A first example of an indented block – in this case, we want to *repeat* this block:

```
1  agedict = {'Zeus': None, 'Denis': 96, 'Alice': 18, 'Rebecca': 20 ,
   ↪  'Bob': 22, 'Cecile': 45}
2
3  myfriends = ['Alice','Bob','Cecile']
4
5  print ("The names and ages of my friends:")
6  for buddy in myfriends:
7  ^^Iprint (f"My friend {buddy} is {agedict[buddy]} years old")
```

Output:

```
1  My friend Alice is 18 years old
2  My friend Bob is 22 years old
3  My friend Cecile is 45 years old
```

What happened here?

```
1 for buddy in myfriends:
2     print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The for loop

1. Take the first element from `myfriends` and call it `buddy` (like `buddy = myfriends[0]`) (line 1)
2. Execute the indented block (line 2, but could be more lines)
3. Go back to line 1, take next element (like `buddy = myfriends[1]`)
4. Execture the indented block ...
5. ...repeat until no elements are left ...

The f-string (*formatted* string)

If you prepend a string with an `f`, you can use `{}` to insert the value of a variable

What happened here?

```
1 for buddy in myfriends:
2     print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a :

More in general, the : and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

What happened here?

```
1 for buddy in myfriends:
2     print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a :

More in general, the : and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

What happened here?

```
1  for buddy in myfriends:
2      print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a :

More in general, the : and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

What happened here?

```
1 for buddy in myfriends:
2     print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a :

More in general, the : and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

What happened here?

```
1 for buddy in myfriends:
2     print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a `:`

More in general, the `:` and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

What happened here?

```
1  for buddy in myfriends:
2      print (f"My friend {buddy} is {agedict[buddy]} years old")
```

The line *before* an indented block starts with a *statement* indicating what should be done with the block and ends with a :

More in general, the : and indention indicate that

- the block is to be executed repeatedly (`for` statement) – e.g., for each element from a list, or until a condition is reached (`while` statement)
- the block is only to be executed under specific conditions (`if`, `elif`, and `else` statements)
- an alternative block should be executed if an error occurs in the block (`try` and `except` statements)
- a file is opened, but should be closed again after the block has been executed (`with` statement)

Can we also loop over dicts?

Sure! But we need to indicate how exactly:

```
1  mydict = {"A":100, "B": 60, "C": 30}
2
3  for k in mydict:    # or mydict.keys()
4      print(k)
5
6  for v in mydict.values():
7      print(v)
8
9  for k,v in mydict.items():
10     print(f"{k} has the value {v}")
```

Can we also loop over dicts?

The result:

```
A
B
C

100
60
30

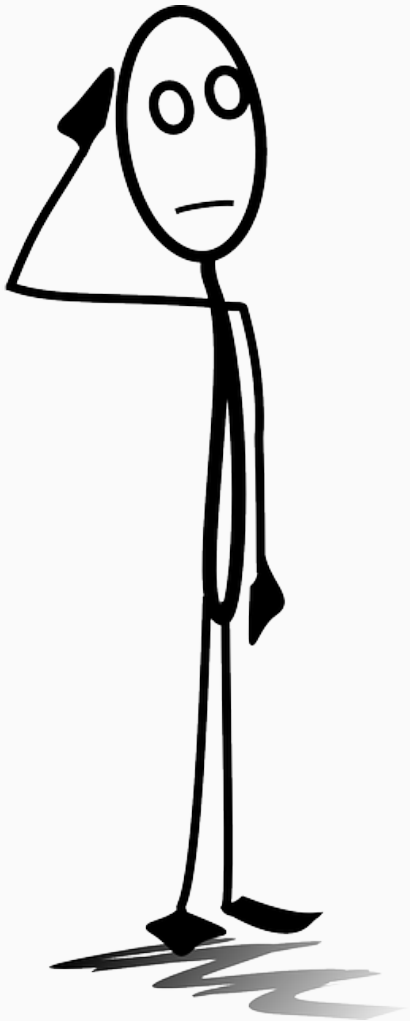
A has the value 100
B has the value 60
C has the value 30
```

if statements

Structure

Only execute block if condition is met

```
1  x = 5
2  if x <10:
3     print(f"{x} is smaller than 10")
4  elif x > 20:
5     print(f"{x} is greater than 20")
6  else:
7     print(f"No previous condition is met, therefore 10<={x}<=20")
```



Can you see how such an if statement could be particularly useful when nested in a for loop?

try/except

Structure

If executed block fails, run another block instead

```
1  x = "5"
2  try:
3     myint = int(x)
4  except:
5     myint = 0
```

Again, more useful when executed repeatedly (in a loop or function):

```
1  mylist = ["5", 3, "whatever", 2.2]
2  myresults = []
3  for x in mylist:
4      try:
5          myresults.append(int(x))
6      except:
7          myresults.append(None)
8  print(myresults)
```

try/except

Structure

If executed block fails, run another block instead

```
1  x = "5"
2  try:
3     myint = int(x)
4  except:
5     myint = 0
```

Again, more useful when executed repeatedly (in a loop or function):

```
1  mylist = ["5", 3, "whatever", 2.2]
2  myresults = []
3  for x in mylist:
4      try:
5          myresults.append(int(x))
6      except:
7          myresults.append(None)
8  print(myresults)
```

Optional

List comprehensions

Structure

A for loop that `.append()`s to an empty list can be replaced by a one-liner:

```
1  mynumbers = [2,1,6,5]
2  mysquarednumbers = []
3  for x in mynumbers:
4      mysquarednumbers.append(x**2)
```

is equivalent to:

```
1  mynumbers = [2,1,6,5]
2  mysquarednumbers = [x**2 for x in mynumbers]
```

Optionally, we can have a condition:

```
1  mynumbers = [2,1,6,5]
2  mysquarednumbers = [x**2 for x in mynumbers if x>3]
```

List comprehensions

Structure

A for loop that `.append()`s to an empty list can be replaced by a one-liner:

```
1 mynumbers = [2,1,6,5]
2 mysquarednumbers = []
3 for x in mynumbers:
4     mysquarednumbers.append(x**2)
```

is equivalent to:

```
1 mynumbers = [2,1,6,5]
2 mysquarednumbers = [x**2 for x in mynumbers]
```

Optionally, we can have a condition:

```
1 mynumbers = [2,1,6,5]
2 mysquarednumbers = [x**2 for x in mynumbers if x>3]
```

List comprehensions

A very pythonic construct

- Every for loop can also be written as a for loop that appends to a new list to collect the results.
- For very complex operations (e.g., nested for loops), it can be easier to write out the full loops.
- But mostly, list comprehensions are really great! (and much more concise!)

$\Rightarrow$ **Useful if you want to explore further; not required for the in-class coding exercise.**

Generators

Structure

A lazy for loop (or function) that only generates its next element when it is needed:

You can create a generator just like a list comprehension (with `()` instead of `[]`):

```
1  mynumbers = [2,1,6,5]
2  squaregen = (x**2 for x in mynumbers)   # these are NOT calculated yet
3  for e in squaregen:
4      print(e)                 # only here, we are calculating the NEXT item
```

Or like a function (but with `yield` instead of `return`):

```
1  def squaregen(listofnumbers):
2      for x in listofnumbers:
3          yield(x**2)
4  mygen = squaregen(mynumbers)
5  for e in mygen:
6      print(e)
```

Generators

Structure

A lazy for loop (or function) that only generates its next element when it is needed:

You can create a generator just like a list comprehension (with `()` instead of `[]`):

```
1  mynumbers = [2,1,6,5]
2  squaregen = (x**2 for x in mynumbers)   # these are NOT calculated yet
3  for e in squaregen:
4      print(e)                 # only here, we are calculating the NEXT item
```

Or like a function (but with `yield` instead of `return`):

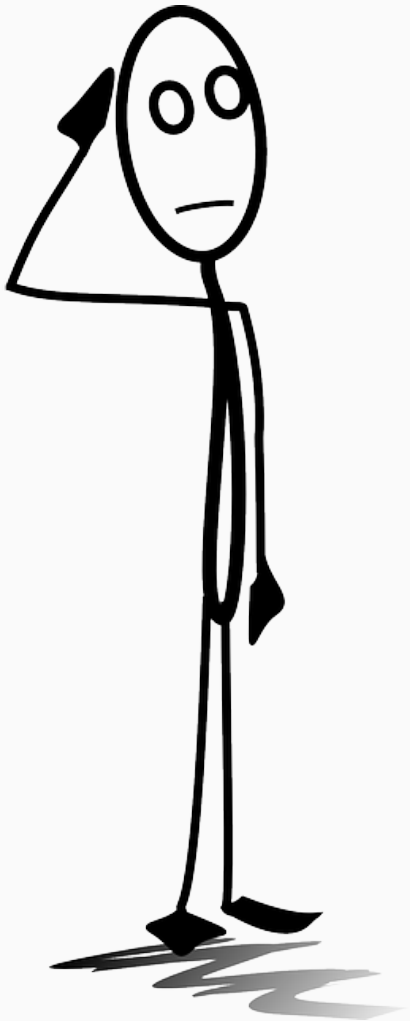
```
1  def squaregen(listofnumbers):
2      for x in listofnumbers:
3          yield(x**2)
4  mygen = squaregen(mynumbers)
5  for e in mygen:
6      print(e)
```

Generators

A very memory and time efficient construct

- Every function that *returns* a list can also be written as a generator that *yields* the elements of the list
- Especially useful if
  - it takes a long time to calculate the list
  - the list is very large and uses a lot of memory (hi big data!)
  - the elements in the list are fetched from a slow source (a file, a network connection)
  - you don't know whether you actually will need all elements

⇒ **You probably don't need this right now, but (a) it will come in very handy once you deal with web scraping or very large collections, and (b) you may come across generators in some examples**



Any questions?

Next steps

Make sure you understood all of today's concepts.

Re-read Chapters 3 and 4 if needed. For now, start praticing with the exercises that you find here: https://github.com/uvacw/teaching-bdaca/blob/main/6ec-course/week01/in-class-exercises.md

Please finish these exercises at home:

https://github.com/uvacw/teaching-bdaca/blob/main/6ec-course/week01/at-home-exercises.md